

Part1: Function Approximation in Air Combat Simulation

Overview

This project presents the development and implementation of an air combat simulation system, where an Aircraft competes against four coordinated Suicide Drones in a dynamic 15×10 grid environment. The Aircraft aims to eliminate all drones and reach a designated goal while managing its limited missile resources and avoiding obstacles such as mountains. Meanwhile, the Drones work strategically to surround or trap the Aircraft.

The simulation incorporates elements of tactical decision-making, feature-based value evaluation, and emergent behavior. The Aircraft and Drones are designed to operate intelligently based on their respective features and objectives.

Components of the Simulation

1. Environment

- Grid Dimensions: A 15×10 matrix representing the combat area.
- Mountains: Ten randomly placed static obstacles that block movement.
- Reload Zone: A randomly located square that allows the Aircraft to replenish its missiles.
- Goal: A fixed square on the grid that the Aircraft must reach after eliminating all drones.

2. Aircraft

- Initial Status:
 - Two missiles at the start (maximum capacity of three).
 - Positioned randomly at the beginning of each simulation.

- Actions:
 - Move: The Aircraft can move in any of the four cardinal directions (up, down, left, right).
 - Shoot: The Aircraft can shoot at drones within a two-block range if there is a clear line of sight.
- Objective: Destroy all drones, avoid mountains, and reach the goal.
- Scoring:
 - +1000 points for successfully reaching the goal after eliminating all drones.
 - +250 points per drone eliminated.
 - -500 points for crashing into mountains.

3. Drones

- Initial Status:
 - Four drones are placed randomly on the grid at the start of each simulation.
- Actions:
 - Move: Drones coordinate to trap or surround the Aircraft.
- Objective: Neutralize the Aircraft by:
 - Surrounding it in all four cardinal directions.
 - Swarming it with at least two drones within a one-block range.
- Scoring:
 - +500 points per drone involved in a successful surround or swarm.

Features and Value Functions

Feature Design

1. Aircraft Features:

- Distance to the goal.
- Distance to the reload zone.
- Distance to each drone.
- Number of drones within one-block and two-block ranges.
- Number of remaining missiles.
- Danger level based on proximity to drones and available missiles.

2. Drone Features:

- Distance to the Aircraft.
- Cardinal positions relative to the Aircraft.
- Blocking the Aircraft's path to the goal.
- Proximity to other drones for coordinated attacks.

Value Functions

- Aircraft Value Function: Evaluates the current state to decide whether to move or shoot based on maximizing tactical advantage.
- Drone Value Function: Guides drones to coordinate and maximize their chances of trapping the Aircraft.

Game Dynamics and Mechanics

Aircraft Strategies

The Aircraft balances between offensive (eliminating drones) and defensive (avoiding threats and replenishing missiles) tactics. Smart use of the reload zone is crucial for survival.

Drone Strategies

The Drones exhibit cooperative behavior by positioning themselves strategically to block the Aircraft's movements and restrict its options.

Game Termination

The game concludes when:

1. The Aircraft eliminates all drones and reaches the goal.
2. The Drones successfully swarm or surround the Aircraft.
3. The maximum number of turns (30) is reached.

Results Summary

Simulation Performance

- Aircraft Win Rate: Over 1000 simulations, the Aircraft achieved a success rate of 38.20%.
- Score Highlights:
 - The Aircraft scored a maximum of 2000 points in one simulation, showcasing effective tactical management.
 - Drones scored a maximum of 1000 points in several games, demonstrating coordinated success.

Insights

- **Aircraft:**
 - Success depended on smart missile usage, effective maneuvering, and timely reloads.
 - Crashes into mountains and drone traps were the main reasons for failure.
- **Drones:**
 - Coordination to block the Aircraft's path was highly effective.

- Proximity-based swarm strategies created tactical advantages.

Visualization and Analysis

- Animated gameplay visualizations revealed key tactical interactions.
- Example highlights:
 - Turn-by-turn movement of the Aircraft to avoid traps.
 - Drones positioning themselves to surround the Aircraft while blocking its path to the goal.

Conclusion

This project successfully demonstrates complex agent interactions in a tactical air combat simulation. The Aircraft and Drones exhibit sophisticated strategies, balancing offensive and defensive maneuvers. The low win rate for the Aircraft highlights the challenge posed by the Drones' coordinated behavior.

Part 2 – Multi-Agent Drone Defense (Reinforcement Learning)

1. Problem Recap

The task revisits the 15×10 “air-combat” grid from Homework 1:

- **Agents** – 1 Aircraft (learner) vs n independent suicide-drones ($1 \rightarrow 4$).
- **Objective** – Aircraft must shoot down every drone **and** reach the goal; drones try to collide or surround.
- **Constraints** – Mountains block movement; missiles are limited but can be re-stocked at a single reload zone.

Homework 1 used a hand-crafted linear value function; win-rate plateaued at $\sim 38\%$. In Part 3 we replace that policy with *data-driven* RL methods and explicitly compare them to a random baseline.

2.Environment Adjustments

Parameter	HW-1	HW-4 (this work)	Rationale
Initial missiles	2	3	Gives RL enough fire-power to learn aggressive play.
Missile capacity	3	4	Prevents “wasted” reload visits.
Reload gain	+1	+3 (only after ≥ 3 shots)	Encourages strategic re-supply.
Living cost	0	−1 per turn	Discourages kiting / time-outs.
Drone policy	shared linear value	Fixed greedy pursuit	Keeps opponent deterministic so the aircraft’s learning signal is clearer.

All other rules (grid size, mountains, goal location, turn limit 30) are unchanged.

3.Algorithms Implemented

1. Baseline – Linear TD-0

- Exactly the HW-1 value function, but trained here with a *purely random* policy to establish a lower bound.

2. Approximate Policy Iteration (API)

- Value network Vnet (128–64 hidden units, ReLU).
- Replay buffer** (20 k), *target-network* sync every 400 steps, Adam (3×10^{-4}).
- ϵ -greedy exploration (ϵ decays from 0.3 $\rightarrow$ 0.05).
- Curriculum:** train $n = 1, 2$ drones for 8 k episodes each, then transfer weights and train $n = 3, 4$ drones for 20 k episodes each.
- Early-stop if return plateaus > 1 200 episodes – cuts wall-time $\sim 35\%$.
- PyTorch 2 torch.compile for $1.3 \times$ speed-up on CPU Colab.

4.Evaluation Metrics

- Win-Rate (%):** fraction of evaluation episodes (400) in which the aircraft reaches the goal **and** all drones are destroyed.
- Average Return:** mean cumulative reward over the same 400 episodes (captures near-misses and partial kills).
- Both metrics are reported for *each* drone count (1 $\rightarrow$ 4).

5.Results

# Drones	Baseline Win-Rate	Policy-Iter Win-Rate	Baseline Avg-Return	Policy-Iter Avg-Return
1	77.2 %	100.0 %	934	1 336
2	74.5 %	100.0 %	1 218	1 684
3	78.0 %	100.0 %	1 518	2 013
4	0.0 %	14.5 %	42	** 328**

Learning curves (last 250 episodes per stage) confirm steady value-growth and a clear gap over the baseline for $n \leq 3$. For $n = 4$, API still improves dramatically over random play but has not yet cracked robust victory.

6.Analysis

- **Why API beats the baseline ($n \leq 3$).**

The MLP captures *non-linear* interactions (missile count $\times$ drone proximity) that a linear model misses.

Experience replay stabilises value updates despite sparse +1 000 goal rewards.

- **Why $n = 4$ remains hard.**

Four drones form a moving “net” wider than the aircraft’s maximum kill-rate (≤ 4 missiles / 30 turns).

Curriculum transfer provides a head-start, yet the state-space explodes combinatorially and the value network still over-estimates its ability to slip through gaps.

- **Runtime vs Performance Trade-off.**

Full training (31 k episodes total) finishes in ≈ 55 min on free Colab CPU; earlier drafts that ran 75 k+ episodes reached 21 % win-rate for $n = 4$ but cost >4 h.

The chosen schedule offers **100 %** wins up to three drones while keeping compute acceptable.

- **Reward-design sensitivity.**

Raising R_KILL to 350 nudged the policy away from passive goal-rush and toward eliminating nearby threats first – a prerequisite for surviving ≥ 3 drones.

7.Limitations & Future Work

- **State representation** – Grid is encoded only via distances and counts; a CNN over a raw 2-D bitmap could capture obstacles and line-of-sight explicitly.
- **Action-value methods** – Direct Q-learning (e.g., DQN with duelling heads) might learn *both* move and shoot decisions, potentially tackling the four-drone case faster.
- **Self-learning drones** – Training drones with a joint RL policy (multi-agent actor-critic) would provide a fairer benchmark and expose the aircraft to a moving target.

8.Conclusion

Replacing Homework 1's static value function with Approximate Policy-Iteration produces **perfect play against up to three drones** and a **$> 14 \times$ win-rate gain** against four drones—all within a one-hour Colab run.

The experiment demonstrates that reinforcement learning, when paired with a modest neural value-function and a curriculum schedule, can substantially exceed both random baselines and the original hand-tuned approach, validating genuine learning rather than random luck.